

Einführung in Deep Learning in der Bildverarbeitung

Prof. Alexander Binder

Week 05: Derivatives I

Task 1 – the example function

Consider this function from the lecture

$$f(x, y) = \begin{cases} \frac{y^3}{x^2+y^2} & \text{if } (x, y) \neq (0, 0) \\ 0 & \text{if } (x, y) = (0, 0) \end{cases}$$

- compute $\delta_{(1,0)}f(0,0) = \frac{\partial f}{\partial x}(0,0)$. This is reallyyyyyy easy, just use the lim-based definition of directional derivative, close your eyes, and receive the present from the Eisleiligen (or almost like that).
- compute $\delta_{(0,1)}f(0,0) = \frac{\partial f}{\partial y}(0,0)$. This is easy again, just use the lim-based definition of directional derivative. It is again a straightforward calculation.
- show that the directional derivatives in $x = (0,0)$ do not satisfy a linear relationship. How ?
 - you got already $\delta_{(1,0)}f(0,0)$ and $\delta_{(0,1)}f(0,0)$.
 - take a simple vector $v = (a, b)$ and compute its directional derivative, using the lim-based definition
 - ... and compare that result against the corresponding linear combination result based on the linear combination of $\delta_{(1,0)}f(0,0)$ and $\delta_{(0,1)}f(0,0)$ which reconstructs v .

Task 2 (Coding): Some derivatives in pytorch

Compute the derivatives in Pytorch for:

$$f(x) = ax^6 + bx^4 + d \ln(x), \quad a, b, d, x \text{ all scalars, } x > 0$$

$$f(x) = e^{-x} \cos(-2x), \quad x \text{ a scalar}$$

$$f(\mathbf{w}) = g(\mathbf{w} \cdot \mathbf{x} + b), \quad g(z) = \frac{1}{1 + e^{-z}}, \quad \mathbf{w} \in \mathbb{R}^d, \mathbf{x} \in \mathbb{R}^d, z \in \mathbb{R}, d = 3$$

$$f(\mathbf{X}) = \mathbf{v} \mathbf{X} \mathbf{a}, \quad \mathbf{X} \in \mathbb{R}^{d \times k}, \mathbf{a} \in \mathbb{R}^{k \times 1}, \mathbf{v} \in \mathbb{R}^{1 \times d}, k = 2, d = 3$$

$$f(\mathbf{X}) = \mathbf{v} \mathbf{A} \mathbf{X} \mathbf{X}^\top \mathbf{B} \mathbf{z}, \quad \mathbf{X} \in \mathbb{R}^{d \times n}, \mathbf{A} \in \mathbb{R}^{d \times d}, \mathbf{B} \in \mathbb{R}^{d \times d}, \mathbf{v} \in \mathbb{R}^{1 \times d}, \mathbf{z} \in \mathbb{R}^{d \times 1}, n = 2, d = 3$$

and for some chosen value of the variables.

Steps:

- define a function (or class with additional parameters as class members) which computes $f(\mathbf{w})$ or $f(\mathbf{X})$. `torch.dot`, `torch.mm` or `torch.einsum` can be of help.

We will also look at `einsum` in the next lecture.

- ask in class if it is not clear for you how to define a class with additional parameters as class members which computes $f(\mathbf{w})$ or $f(\mathbf{X})$
- it has some other parameters, such as v, a, A, B, z, x . For these parameters, seed random generators, then draw the tensors from a zero mean normal distribution with unit covariance or another distribution, in order to set them to some values. You can do the random number generation in PyTorch, numpy is not strictly necessary.
- make sure that the output of the class/function is a scalar, call `backward`, [print the gradient](#)

Task 3 (Coding): A second derivative in PyTorch

This is actually not that hard. You get a step-by-step instruction with the correct solution to check the computed gradients.

The high-level plan:

- compute the first order gradient (`torch.autograd.grad` or `backward`)
- if using `backward`, clone the `inputtensor.grad` field and **set it to None (!)**
- you got the vector as gradient, compute now the second derivative by calling one of the above methods (`torch.autograd.grad` or `backward`) on $\nabla f|_x \cdot w$ for some weight tensor w

The concrete steps:

- implement for a vector x of shape (n) and a matrix A of shape (n, n) the function

$$y = x^\top Ax$$

It should return a vector y of shape (n) (not $(n, 1)$ or $(1, n)$) and take x , A as inputs. Note: this is the generalization of ax^2 for vectors.

This is a good exercise to use `@ / matmul` and `unsqueeze / squeeze`.

- create x , A using `torch.randn` as random values (or another random number generator), it makes sense to use a small $2 \leq n \leq 10$.
- steps to get the first and the second derivative:
 - we want derivatives for x , so set the `requires_grad` field of this tensor. No need to do this for A .
 - call the function to get y
 - either use `torch.autograd.grad(...)` to get the first derivative or `y.backward(create_graph=True)`. The first version is more clean, actually, the second you have seen how to use it.
 - if you used `y.backward(create_graph=True)`, you have to do this:
 - * clone `x.data` into a tensor named `deriv1` or `grad1` or the like
 - * set `x.data` to `None`
 - you have now the first order gradient tensor in `deriv1`.
 - verify correctness by computing the norm of the difference ($\|a - b\|$) between `deriv1` and the theoretically correct solution `(x.unsqueeze(0) @ (A + A.T)).squeeze()`.

This corresponds to

$$\begin{aligned} f(x) &= x^\top Ax \\ \nabla f(x) &= x^\top (A + A^\top) \\ &\sim (A + A^\top)x \end{aligned}$$

It should be on the order what is typical as error for float32: 10^{-6} to 10^{-7}

- You cannot compute the derivative of a vector $\nabla f|_x$, you need a scalar.

however you can compute the derivative of $\nabla f|_x \cdot w$ for any vector w . Note that if $w = (0, \dots, 0, \underbrace{1}_{\text{at } i}, 0, \dots, 0)$ is a one-hot-vector,

$$\text{then } \nabla f|_x \cdot w = \nabla f|_x \cdot (0, \dots, 0, \underbrace{1}_{\text{at } i}, 0, \dots, 0) = \frac{\partial f}{\partial x_i}|_x$$

- get the second derivative:
 - * get a tensor w of shape (n)
 - * either compute `torch.autograd.grad(...)` for `deriv.unsqueeze(0) @ w.unsqueeze(1)`
 - * or `deriv.backward(w)`
 - * if used backward, clone `x.grad` as second derivative
 - verify that the resulting tensor, say `deriv2` is approximately the value of `(w.unsqueeze(0) @ (A + A.T)).squeeze()` by computing the norm of the difference to that value.
 - **error source:** if you do not set `x.data = None` when using `backward`, you would get an accumulation of first and second order gradients as a weird mistake. That's why `torch.autograd.grad(...)` is the cleaner way for second derivatives
 - done! that simple
- you can try this also for second derivatives of one-dimensional functions say $f(x) = 2x^3 - 5x^2 + 3x - 7$